2026课程设计选题

本学期的课程设计将会分为两个阶段:

上半学期

- **目标**：着重探讨**代码编写**和**策略搜索**，主要涵盖策略类和博弈类游戏。这个阶段的学习要求先掌握并实现基本的游戏规则，然后通过自己的实践和经验来设计游戏策略，让程序能够自动地进行游戏。
- **选题**：任选一题完成阶段一任务。
- **截止时间**：2026.4.30 20:00
- **提交材料**：
  - 源代码：提交代码推荐包含可执行指令以及路径（不要使用绝对路径），或可执行文件，“一键执行”；
  - Readme文件：用来说明运行源代码所需要执行的命令，环境依赖等；
  - 课程设计报告：包含选题描述、方案设计、代码模块的功能划分与描述、实现效果等。推荐使用markdown文件，并在报告中体现每个功能的gif演示。推荐用精简的语言介绍代码结构，算法原理只需指出哪部分代码实现的是什么算法，不需要讲具体细节。

下半学期

- **目标**：**结合课程所学的人工智能知识来解决实际任务**。针对选题，利用所学知识编写AI来进行游戏，将所学知识应用于实际问题。
- **选题**：针对上半学期选题，完成阶段二任务。
- **截止时间**：2026.6.1 20:00
- **提交材料**：
  - 源代码：提交代码推荐包含可执行指令以及路径（不要使用绝对路径）、或可执行文件，程序可“一键执行”；
  - Readme文件：用来说明运行源代码所需要执行的命令，环境依赖等；
  - 课程设计报告：包含选题描述、方案设计、代码模块的功能划分与描述、实现效果等。推荐使用markdown文件，并在报告中体现每个功能的gif演示。

注：允许自行拟定选题，但需满足以上课程要求，并与助教老师商量。

---

1. 微型文明：资源经营、战略规划与智能决策【姜锦瑞】

背景介绍：

《文明》类游戏是一类典型的策略经营游戏。玩家通常需要在地图上建立城市、采集资源、发展科技，并在有限回合内不断提升整体文明水平。这类问题具有明确的状态、规则、资源约束和决策目标，适合用程序设计方法进行抽象，也适合进一步引入启发式策略、搜索规划等人工智能方法。

本题要求同学们设计一个“微型文明”环境。为保证可实现性，本题不要求实现复杂战斗系统、外交系统或庞大的科技树，而是聚焦于“地图—资源—城市—科技—决策”这一最核心的闭环。通过本题，同学们将训练环境建模、面向对象程序设计、规则实现、状态空间搜索与智能体设计等方面的能力。

基本环境设定

1. 地图默认为 10×10 栅格地图，也可支持 8×8～12×12 的自定义大小。
2. 地图包含以下地形类型：
    - 平原：每回合提供 1 单位粮食，可建城
    - 森林：每回合提供 1 单位木材，可建城
    - 山地：每回合提供 1 单位矿石，不可建城
    - 河流：每回合提供 1 单位粮食和 1 单位科技点，可建城
    - 荒地：不提供资源，不可建城
3. 游戏开始时，系统自动生成地图，并放置 1 座初始城市（首都）。
4. 新城市只能建立在平原、森林或河流格上，且任意两座城市中心之间至少间隔 3 格。
5. 每座城市会占据其周围 1 格范围内的区域，用于计算该城市的基础资源产出。
6. 游戏总回合数默认为 30 回合。

资源与科技设定

1. 游戏中至少包含以下资源：
    - 粮食
    - 木材
    - 矿石
    - 科技点
2. 科技树至少包含以下四项：
    - 农业（解锁农场）
    - 伐木（解锁伐木场）
    - 采矿（解锁矿场）
    - 教育（解锁图书馆）
3. 推荐科技花费如下：

- 农业：6 科技点
- 伐木：6 科技点
- 采矿：8 科技点
- 教育：10 科技点

建筑设定

每座城市可建设的建筑至少包括：

- 农场：消耗 2 木材，提供 +2 粮食/回合（需农业）
- 伐木场：消耗 2 木材，提供 +2 木材/回合（需伐木）
- 矿场：消耗 2 木材 + 1 矿石，提供 +2 矿石/回合（需采矿）
- 图书馆：消耗 2 木材 + 2 矿石，提供 +2 科技点/回合（需教育）

每种建筑在同一城市中至多建造一次。

回合流程

每回合建议按照如下顺序进行：

1. 玩家或 AI 选择一个动作
2. 执行动作
3. 所有城市结算本回合资源产出
4. 回合数加一

动作可至少包括：

- 新建城市
- 在指定城市中建造建筑
- 研究一项科技
- 跳过本回合

终局目标

在限定回合数内最大化文明总分。建议总分由以下部分构成：

- 城市数量
- 已建成建筑数量
- 已解锁科技数量
- 剩余资源数量

可参考如下评分函数：

$$Score = 20 \times \text{城市数} + 5 \times \text{建筑数} + 8 \times \text{科技数} + \left\lfloor \frac{\text{粮食} + \text{木材} + \text{矿石} + \text{科技点}}{4} \right\rfloor$$

阶段一：环境搭建与规则驱动的经营决策

阶段一任务：

搭建“微型文明”游戏环境，完成基础规则实现与交互流程。

1. 实现地图生成与地图显示。
2. 实现城市、资源、科技、建筑等核心对象及其规则。
3. 实现完整回合制流程。
4. 实现动作合法性判断，例如：
   - 不能在非法地块建城
   - 资源不足时不能建造建筑或研究科技
   - 已建成的建筑不能重复建造
5. 实现文明总分计算。
6. 可视化界面可采用命令行实现；有能力的同学可进一步使用 Tkinter、PyQt、Pygame 等工具实现 GUI。

实现目标：实现基于规则或贪心策略的基础经营 AI

你的程序需要实现一个 baseline 智能体，使其能够在无人干预的情况下自动经营一个文明，并尽可能获得较高分数。baseline AI 至少应具备以下能力：

1. 能根据资源情况选择优先发展的方向，例如优先补足粮食、优先积累科技点或优先扩张城市。
2. 能在多个合法建城位置中选择一个相对更优的位置。
3. 能在已有城市中选择较合理的建筑顺序。
4. 能在若干科技中选择下一项研究目标。

baseline 智能体实现提示：

为保证阶段一任务具有明确的实现目标，建议同学们首先完成一个基于规则或贪心策略的 baseline 智能体。baseline 的目的在于：

1. 验证环境与规则实现是否正确；

2. 为阶段二的搜索规划智能体提供对照；

3. 作为“可运行、可比较、可复现”的基础方案。

> 建议的 baseline 实现思路如下：
>
> 1. **建城决策**

> - 枚举所有合法建城位置；
> - 对每个位置计算一个简单评分，例如统计该位置周围若干格内可获得的粮食、木材、矿石、科技点总量；
> - 优先选择综合资源潜力较高的位置建城。
>
> 2. **建筑决策**
> - 当某类资源产出明显不足时，优先建造对应的增产建筑；
> - 例如：
> - 粮食偏低时优先建造农场；
> - 木材偏低时优先建造伐木场；
> - 科技点偏低时优先建造图书馆。
>
> 3. **科技决策**
> - 若某类建筑尚未解锁且当前文明发展明显受限于该类资源，则优先研究对应科技；
> - 例如：
> - 缺乏粮食增产手段时优先研究农业；
> - 缺乏科技点时优先研究教育。
>
> 4. **动作优先级**
> - 可按照“资源补短板 > 高收益建城 > 关键科技 > 其他建筑 > 跳过”的顺序决策；
> - 也可以自行设计等价的规则体系，但需在报告中说明理由。
>
> 5. **最低要求**
> - baseline 智能体应保证在任何合法状态下都能输出一个合法动作；
> - 不允许仅使用纯随机策略作为最终 baseline；
> - 允许先实现随机策略用于调试环境，但提交版本应体现基本的规则性或贪心思想。

说明：

- baseline 不要求最优，也不要求使用复杂搜索算法；
- 其核心目标是“具备基本经营逻辑，并能稳定优于随机策略”；
- 建议在实验报告中比较随机策略与 baseline 智能体在平均得分上的差异。

阶段二：搜索规划与智能决策

背景介绍——规划问题：

在策略经营类游戏中，当前回合的决策往往会影响未来多个回合的收益。例如，本回合是优先建造图书馆，还是优先扩张城市；是先研究农业，还是先研究教育，这些都属于典型的序列决策问题。为了

做出更优决策，我们需要在当前状态下预测若干步未来，并比较不同动作序列可能带来的结果。

阶段二任务：

实现目标：实现一种面向未来若干回合的搜索规划智能体

1. 在阶段一环境的基础上，实现一种搜索规划方法，用于在当前状态下选择更优动作。可选方法包括但不限于：
    - Beam Search（束搜索）
    - MCTS（蒙特卡洛树搜索）
    - Rolling Horizon Search（滚动时域搜索）
2. 搜索深度或规划步数建议设置为 3～5 步。
3. 需要设计状态评估函数，用于估计某个中间状态的优劣。
4. 需要对动作空间进行适当裁剪，避免搜索空间过大。例如：
    - 只保留若干个评分最高的建城位置
    - 只扩展收益较高的建筑动作
    - 在资源明显不足时不扩展无效动作
5. 在实验报告中说明：
    - 选择了哪一种搜索方法
    - 状态评估函数如何设计
    - 动作裁剪策略如何设计
    - 搜索深度与运行时间之间的关系

实验要求：

1. 至少比较以下三种策略的表现：
    - 随机策略
    - 阶段一的 baseline 规则 AI
    - 阶段二的搜索规划 AI
2. 建议在多张固定随机种子生成的地图上进行实验，并给出平均分数。
3. 建议从以下角度分析结果：
    - 最终得分
    - 各类资源积累情况
    - 城市扩张数量
    - 科技发展情况
    - 单步决策时间

选做探索方向：

1. 尝试让系统支持自然语言形式的战略目标，例如：
    - “优先科技发展，谨慎扩张”
    - “前期优先保证粮食和木材，后期冲高分”
2. 若选择该方向，可使用大语言模型将自然语言目标转化为评分函数的权重或规则参数，再与阶段二搜索规划模块结合。
3. 选做部分不作为必做要求。

实现建议：

- 本阶段的重点不在于把地图做得很复杂，而在于把状态表示、动作设计、搜索规划和评估函数做好；
- 搜索方法不需要追求“大而全”，只要能在小规模环境中稳定优于 baseline AI，即可体现其效果；
- 若实现 GUI，建议将 GUI 与游戏逻辑解耦，避免影响搜索模块的调试与复用。

2. 五子棋博弈【姜锦瑞】

阶段一：五子棋环境构建与规则引擎

背景介绍：
五子棋起源于中国，是一种经典的两人对弈棋类游戏。双方分别使用黑白两色棋子，轮流落子在棋盘竖线与横线的交叉点上，先形成五子连珠者获胜（横线、竖线、斜线均可）。本题要求同学们先完成一个可运行的五子棋程序，在此基础上实现基础规则引擎与简单智能体，从而训练状态表示、规则实现、合法性判断、胜负判定和基础交互能力。

阶段一任务：
搭建五子棋游戏环境，完成基础规则实现与对弈流程设计。

1. 实现可正常运行的五子棋程序，并支持完整对弈流程。
2. 默认采用标准 15×15 棋盘，同时应支持自定义棋盘大小。
3. 在游戏开始时，能够确定先手方；可采用固定黑棋先手，也可随机指定黑棋或白棋先行，但需在程序中说明规则。
4. 正确维护棋盘状态，区分空位、黑棋与白棋。
5. 支持按照玩家输入的位置进行落子，并实现以下判定：
    - 边界检测
    - 当前位置是否已被占用
    - 落子是否合法
6. 若玩家输入的位置不合法，程序应给出明确提示，并要求重新输入。
7. 实现连五胜负判定，能够判断横向、纵向、主对角线和副对角线上的五连。

8. 可选实现平局判定：当棋盘填满且双方均未获胜时，判定为平局。
9. 本阶段不强制实现三三禁手、长连禁手等复杂 Renju 规则。
10. 游戏界面可采用命令行实现；有能力的同学可进一步使用 Tkinter、PyQt 等工具实现 GUI，支持鼠标点击落子等交互方式。

实现目标：实现基础对弈与简单智能体

1. 至少支持以下两种模式中的一种：
   - 双人对战
   - 人机对战
2. 若实现人机对战，应支持玩家与 AI 轮流落子，并能够完整进行至游戏结束。
3. 实现一个极简 baseline 智能体，用于完成基础人机对弈。

baseline 智能体可选思路包括：

- 随机选择一个合法位置落子
- 优先在已有棋子附近落子
- 基于简单规则进行防守或封堵

实现提示：

- 设计 baseline 智能体时，应尽量保证其“任何时刻都能给出一个合法落子位置”，避免因规则不完备而无法继续运行。
- 若采用规则式 AI，建议优先考虑封堵对手的连续威胁，再在必要时尝试扩展自身棋形。
- 可以考虑的基础棋形包括：连二、连三、单四等；若检测到对方存在明显威胁，应优先进行防守。
- 若尝试使用 BFS 或 DFS 等搜索方法实现基础 AI，应注意搜索空间迅速增长的问题，并在报告中分析搜索深度与运行时间之间的关系。

实现建议：

- 棋盘状态建议使用二维数组或矩阵表示。
- 建议将“棋盘表示”“落子合法性判断”“胜负判定”“AI 决策”划分为相对独立的模块，便于后续阶段继续扩展。
- 若使用 GUI，建议优先保证规则判断正确，再进一步优化交互体验。

阶段二：五子棋高级搜索博弈算法与智能对战

背景介绍：

本题是阶段一“五子棋环境构建与规则引擎”的自然延续，重点对应课程后半段中“搜索、规划、规则推理”这一核心训练目标。在阶段一完成的五子棋程序基础上，本阶段要求进一步实现启发式局面

评估、博弈树搜索、剪枝优化与实验分析，使系统从“能够完成对弈”提升为“具备一定棋力与决策依据”。

背景介绍——博弈树：
博弈树（Game Tree）是人工智能中研究双人对抗决策问题的重要工具。树中的每一个节点都对应一个棋盘局面，边表示一次可能的合法落子。对于五子棋而言，我们的目标是在当前局面下，从所有可能的下一步中选择最优落子点。

因此，本阶段需要建立两类核心机制：

1. **价值评估函数（Critic）**：对当前棋盘局面进行量化评分。该评分函数需要能够反映局面对某一方的有利程度。例如，若评分越高表示对黑方越有利，则评分越低表示对黑方越不利。
2. **策略选择函数（Actor）**：在候选落子中选择一个最优动作，通常与博弈树搜索、贪婪选择或其他搜索策略结合使用。

阶段二任务：

实现目标：实现启发式评估函数与基础博弈树搜索框架

1. 设计静态局面评估函数，对当前局面进行打分。
2. 评估函数中可考虑的典型棋形包括：
    - 连二
    - 连三
    - 活三
    - 冲四
    - 活四
    - 连五
3. 在实验报告中说明：
    - 评估函数考虑了哪些棋形模式
    - 各类棋形的权重如何设定
    - 为什么采用这样的权重设置
4. 必须实现以下两项内容：
    - Minimax 搜索
    - Alpha-Beta 剪枝
5. 不接受仅提交裸暴力搜索程序。
6. 程序应能够在给定局面下，基于搜索结果输出推荐落子位置，并给出对应的评估分数或胜率估计。

实现目标：完成搜索优化与实验分析

1. 在基础搜索算法上，至少叠加一种启发式优化策略，例如：

- 仅在已有棋子邻域内生成候选点
- 启发式节点排序（Move Ordering）
- 深度裁剪
- 优先扩展高分候选位置

2. 在实验报告中必须给出优化前后的量化比较结果，至少包括：
   - 节点遍历数
   - 可达到的搜索深度
   - 单步决策耗时（可作为辅助指标）
3. 比较以下两类智能体的实际表现：
   - 阶段一中的 baseline 智能体
   - 本阶段实现的启发式搜索智能体
4. 建议给出若干典型对局案例，并分析 AI 在关键位置的决策依据。

进阶探索方向（任选其一，供学有余力的同学参考）：

方向一：MCTS（蒙特卡洛树搜索）

1. 实现蒙特卡洛树搜索，并在限定思考时间下与 Alpha-Beta 方法进行比较。
2. 比较内容可包括：
   - 单步耗时
   - 节点扩展规模
   - 对弈胜率
   - 在不同棋盘规模下的稳定性

方向二：数据驱动的价值评估函数

1. 在启发式评估函数之外，尝试使用数据驱动方法学习局面评分函数。
2. 可选方式包括：
   - 基于棋谱数据训练回归或分类模型，例如 Logistic 回归、支持向量机、K 近邻、卷积神经网络等
   - 将模型输出作为局面评分函数的一部分，与搜索算法结合
3. 若采用该方向，建议在报告中说明：
   - 数据来源与清洗方式
   - 特征表示方法
   - 模型训练与评估方法

    - 与手工设计评估函数相比的优缺点
4. 可通过旋转、翻转等方式对棋谱数据进行增强。

方向三：强化学习方法（类似 AlphaZero）

1. 在 MCTS 基础上，尝试构建策略价值网络。
2. 实现五子棋自我对弈，用于采集训练数据。
3. 数据中应尽量包含：
    - 当前棋盘状态
    - 各候选动作的策略分布
    - 最终对局结果
4. 可采用“自我对弈—更新模型—再自我对弈”的迭代方式进行训练。
5. 建议：
    - 策略价值网络可采用共享卷积层后接策略头与价值头的结构
    - 可通过观察损失函数或对弈表现决定训练轮次
    - 可使用数据增强方法提高样本利用效率

方向四：LLM 对战实验

1. 将棋盘状态转化为结构化文本提示，调用大模型进行落子决策。
2. 若选择该方向，必须记录并分析：
    - API 调用成功率
    - 平均每步 Token 消耗
    - 非法落子率
    - 单步延迟
    - 与经典搜索算法在棋力上的差异
3. 需要特别说明：
    - LLM 路线仅建议作为探索性对照实验，不建议作为主实现路径
    - 应重点分析大模型在棋盘状态理解、规则遵守与稳定决策方面的局限性

实现建议：

- 搜索空间非常大，应结合启发式候选点生成方法减少分支数。
- 静态评估函数的设计会直接影响搜索效果，建议优先做好棋形模式识别。
- Alpha-Beta 剪枝的效果通常依赖较好的节点排序策略。
- 若涉及数据驱动方法，应注意数据清洗、异常样本处理与过拟合问题。

- 实验部分不应只给出结论，应尽量通过量化指标比较不同方法与优化策略的收益。

附件：

五子棋棋谱.zip
1.63MB

大家也可以自行寻找棋谱。

3. 炸弹人【 潘勉之 】

阶段一：炸弹人环境构建与规则引擎

- **背景介绍：** 课题要求实现一个基于网格的回合制“炸弹人”双人博弈游戏（炸弹人游戏简介）。地图由空地、不可摧毁的铁墙、可摧毁的木箱，以及**随机游走的怪物**组成。双方玩家（与怪物）在局内轮流行动。玩家每回合可选择移动一步（上/下/左/右）或在原地放置一颗几回合后呈十字形爆炸的炸弹。若玩家被炸弹波及，**或与怪物发生碰撞（处于同一格子）**，将直接判负。 本题要求同学们先完成一个可运行的炸弹人程序，在此基础上实现基础规则引擎与简单智能体，从而训练状态表示、规则代码化实现、合法性判断、胜负判定和基础交互能力
- **实现目标1：搭建环境与基础规则**
  - **棋盘大小与实体初始化：**
    - **尺寸设定：** 默认采用标准的 `11 × 11` 奇数网格棋盘，同时应**支持自定义棋盘大小**。
    - **地图生成：** 所有横纵坐标均为奇数（从0开始计数）的格子固定为铁墙；在空地中以一定概率随机生成木箱。
    - **玩家初始化：** 在游戏开始时，能够确定先手方。玩家 1 固定在左上角 `(0, 0)`，玩家 2 固定在右下角 `(10, 10)`。必须保证玩家出生点及其直接相邻的格子强制为空地（安全区）。
    - **怪物初始化：** 在远离玩家出生点的空地或木箱中，随机生成数只怪物。
  - **状态表示与动作空间：**
    - **底层数据结构：** 棋盘状态建议使用二维数组或矩阵表示。正确维护棋盘状态，需使用不同数值区分空位、玩家 1、玩家 2、铁墙、木箱、怪物以及不同倒计时的炸弹。
    - **玩家动作空间：** 在每个回合中，玩家的合法输入严格定义为 6 种离散动作之一：1.向上移动；2.向下移动；3.向左移动；4.向右移动；5.原地放置炸弹；6.原地待命。
  - **合法性判断与核心规则引擎：** 支持按照玩家输入的指令进行操作，并实现以下核心判定：
    - **移动与边界检测：** 移动位置是否超出边界，目标位置是否已被铁墙、木箱、未爆炸弹占据。若玩家输入的位置不合法，程序应给出明确提示，并要求重新输入。

    - **怪物行动逻辑**：怪物在两名玩家回合结束后统一行动（即每轮行动顺序为：玩家1 -> 玩家2 -> 所有怪物）。怪物每回合只能在相邻的空地随机移动 1 格（不可穿墙或木箱）。
    - **致命碰撞判定：若玩家主动走入有怪物的格子，或怪物在回合中走入玩家所在的格子，该玩家立即死亡（判负）。**
  - **炸弹爆炸结算与波及范围说明**： 炸弹放置后进入倒计时（如 3 个回合后爆炸）。爆炸范围与结算规则必须严格遵循以下逻辑：
    - **十字形爆炸范围**：炸弹爆炸时，以炸弹所在坐标为中心，向正上、正下、正左、正右四个方向延伸，**默认爆炸半径（范围）为 2 格**。即如果不受任何阻挡，爆炸将覆盖一个包含 9 个格子的十字形区域。
    - **爆炸阻挡与破坏**：
      - 火焰遇到**空地**：正常蔓延。
      - 火焰遇到**铁墙**：立刻阻断，铁墙完好无损，且火焰不再向该方向的后续格子延伸。
      - 火焰遇到**木箱**：将木箱摧毁（变为空地），并在此格子阻断延伸。
    - **爆炸击杀（含怪物）**：处在十字形爆炸范围内的**所有玩家和怪物都会被炸死**。怪物死亡后从棋盘上移除；玩家死亡触发胜负结算。
  - **胜负与平局判定**：
    - 判定标准：实现被炸弹波及或被怪物碰头的判定，触发者判负。
    - 平局：可选实现平局判定，当由于同一颗炸弹爆炸导致双方同时死亡，或棋盘填满/超过最大回合数且双方均存活时（也可以加额外的规则，例如均存活时根据杀死的怪物数判定胜负），判定为平局。
  - **交互界面设计**： 游戏界面可采用命令行实现；有能力的同学可进一步使用 Tkinter、PyQt 等工具实现 GUI，支持通过键盘方向键或按钮等交互方式操作
- **实现目标二：基础对弈与简单智能体**
  - 至少支持双人对战与人机对战中的一种；若实现人机对战，应支持玩家与 AI 轮流行动，并能够完整进行至游戏结束。
  - **极简 Baseline 智能体**：实现一个初级 AI 用于人机对弈。可选思路包括：随机选择合法的移动或放炸弹指令；优先向没有怪物和即将爆炸炸弹的绝对安全区移动。
  - **实现建议**：设计 baseline 智能体时，应尽量保证其“任何时刻都能给出一个合法落子（行动）位置”，避免因规则不完备而无法继续运行。强烈建议将“棋盘表示”、“行动合法性判断”、“状态结算”、“AI决策”划分为相对独立的模块，便于后续阶段继续扩展。

阶段二：高级对抗搜索博弈算法与大模型对战

- 本题是阶段一“炸弹人环境构建与规则引擎”的自然延续。在阶段一完成的对弈程序基础上，本阶段要求进一步实现启发式局面评估、博弈树搜索、剪枝优化与实验分析，使系统从“能够完成对弈”提升为“具备一定棋力与决策依据”。

- **博弈树（Game Tree）引入：** 博弈树是人工智能中研究双人对抗决策问题的重要工具。树中的每一个节点都对应一个迷宫局面（包含双方玩家、怪物位置、炸弹倒计时等状态），边表示一次合法的行动（Action）。我们的目标是在当前局面下，从所有可能的下一步中选择最优动作。因此，本阶段需要建立两类核心机制：
  - **价值评估函数（Critic）**：对当前迷宫局面进行量化评分，反映局面对某一方的有利程度。
  - **策略选择函数（Actor）**：在候选动作中选择一个最优动作，通常与博弈树搜索结合使用。
- **实现目标一：启发式评估函数与基础博弈树搜索（核心算法要求）**
  - **设计静态局面评估函数：** 设计函数对当前局面进行打分。在“炸弹人与怪物”的环境中，评估函数可考虑的典型特征包括：
    - **绝对安全性**：当前位置是否在炸弹十字爆炸范围内？是否与怪物距离过近？
    - **机动性（活动空间）**：角色周围的“安全空地”数量（避免被炸弹或怪物逼入死胡同）。
    - **攻击性**：放置的炸弹对敌方生存空间的压缩程度，或炸毁木箱带来的收益。
    - **报告要求**：必须在实验报告中说明评估函数考虑了哪些特征模式，各类特征的权重如何设定，以及为什么采用这样的权重设置。
  - **基础与进阶搜索算法实现：**
    - **Minimax 搜索**：实现经典的极小极大搜索算法，模拟双方的最佳应对。
    - **Alpha-Beta 剪枝**：在 Minimax 基础上实现 Alpha-Beta 剪枝以提升搜索效率。
- **实现目标二：完成搜索优化与量化分析（进阶算法要求）**
  - **搜索优化策略：** 搜索空间非常大，应结合启发式候选点生成方法减少分支数。在基础搜索算法上，**至少叠加一种**启发式优化策略，例如：
    - **启发式节点排序**：Alpha-Beta 剪枝的效果通常依赖较好的节点排序策略。例如，优先扩展“移动到绝对安全区”或“放置炸弹”的高分动作。
    - ……
  - **量化对比实验：** 在实验报告中**必须给出优化前后的量化比较结果**，至少包括：
    - 节点遍历数
    - 可达到的搜索深度
    - 单步决策耗时（可作为辅助指标）
  - **对局案例分析：** 比较阶段一中的 baseline 智能体与本阶段实现的启发式搜索智能体的实际表现。建议给出若干典型对局案例（如 AI 是如何利用延迟爆炸把对手或怪物逼死的），并分析 AI 在关键位置的决策依据。
- **实现目标三：大语言模型（LLM）作为对手的探索实验（核心亮点）**

  本方向要求将大语言模型（LLM）引入博弈系统，测试其在二维空间规划和规则理解上的能力。

- **实验设计**： 将当前回合的棋盘状态（可简化为玩家视野内的局部网格）转化为结构化文本提示（Prompt），调用大模型API进行决策，让 LLM 输出下一步的 Action，并让其作为对手与你的经典搜索 AI 进行对战。
- **量化分析要求**： 若实现该方向，在报告中记录并分析（不限于）以下指标：
  - **API 调用成功率**与**平均每步 Token 消耗**。
  - **非法行动率**（例如 LLM 经常输出撞墙、主动走到怪物脸上）。
  - **与经典搜索算法（Alpha-Beta）在生存率/胜率上的差异**。
- *(注：LLM 路线仅建议作为探索性对照实验，不建议作为替代 Minimax 的主实现路径。学有余力的同学也可额外探索 MCTS 蒙特卡洛树搜索或强化学习自我对弈方向。)*

4. 校园生活日程规划与成就养成【 潘勉之 】

游戏介绍

在快节奏的大学生活中，时间安排、体力管理、饮食节奏、学习与锻炼之间存在复杂权衡。本课程设计以“校园多智能体日程规划游戏”为载体，让学生在一个具有时间约束、空间约束、资源容量约束和状态反馈的仿真环境中，完成从游戏规则设计到智能算法实现的完整实践。

基础环境设定与核心规则

- 时间系统与地图空间
  - 游戏总时长为 7 天，每天 24 小时，共 168 个离散时间步。每个时间步代表 1 小时。系统需维护当前日期、小时和各智能体的状态变化，并在每个时间步完成动作执行、属性结算和日志记录。
  - 校园地图采用二维网格地图或连通图表示。地图中至少包含以下 5 类核心建筑：学生宿舍、教学楼、校园食堂、图书馆、体育馆。
- 建筑初始位置与距离规则
  - 可以统一采用 10×10 网格地图，并为核心建筑设置固定坐标。一个参考示例如下：

建筑	推荐坐标	说明
学生宿舍	(1, 1)	学生智能体的默认初始位置
教学楼	(2, 7)	主要用于“上课”动作
校园食堂	(5, 3)	主要用于“进食”动作
图书馆	(8, 8)	主要用于“自习”动作

体育馆	(9, 2)	主要用于“运动”动作

  - 也可以采用全连通图设计地图，需要定义任意两个建筑之间的距离
  - 若采用网格地图，建筑间距离推荐使用曼哈顿距离：$d(A,B)=|x_A-x_B|+|y_A-y_B|$。为避免移动时间过大、压缩有效行动回合，定义建筑 A 到建筑 B 的移动耗时为：$t_{move}(A,B)=max(1,ceil(d(A,B)/3))$。也就是说，平均跨越约 3 个网格需要 1 小时。
- 智能体只有在移动完成后的下一个时间步，才能在目标建筑执行对应动作。移动除了消耗对应时间外，还需消耗体力与饱食度。推荐设置为：体力消耗 = 2 × t_move，饱食度消耗 = 1 × t_move。若智能体处于“疲劳”状态，则每次移动耗时额外增加 1 小时。
- 建筑开放时间与人数容量限制
  - 学生宿舍：24 小时开放，容量不限。
  - 教学楼：08:00 - 18:00 开放，容量不限。
  - 校园食堂：07:00 - 09:00、11:00 - 13:00、17:00 - 19:00 开放，容量不限；同一时段在食堂用餐的智能体越多，社交氛围越强，每多 1 名同场智能体可额外获得 +1 心情值。
  - 图书馆：08:00 - 22:00 开放，容量上限为 2 人。
  - 体育馆：10:00 - 21:00 开放，容量上限为 1 人。
- 系统必须检查非法动作。若智能体尝试进入未开放建筑，或目标建筑在到达时已满员，则该动作视为无效动作，系统应拦截该行为并给出清晰提示；学生也可以进一步设计“等待”“改道”或“重新规划”等策略。

智能体状态、动作收益与 Debuff 规则

- 每名学生智能体都具有基础资源属性与收益属性。基础资源包括体力（上限 100）和饱食度（上限 100）；收益属性包括学分、健康值和心情值。智能体在建筑中停留并执行 1 小时动作时，按下表结算：

动作	地点	体力变化	饱食度变化	收益
睡眠	宿舍	+15	-2	无直接分数收益
进食	食堂	0	+30	+5 心情值
自习	图书馆	-8	-5	+10 学分
上课	教学楼	-10	-5	+15 学分，-2 心情值
运动	体育馆	-15	-10	+10 健康值，+5 心情值

- 当体力或饱食度降至 0 及以下时，触发 Debuff：
  - 疲劳（体力 ≤ 0）：无法执行“自习”“上课”“运动”等高消耗动作；移动耗时增加；心情值每小时额外 -2。必须回宿舍睡眠后解除。
  - 饥饿（饱食度 ≤ 0）：在图书馆或教学楼的学分收益减半（向下取整）；健康值每小时 -2。必须前往食堂进食后解除。

初始天赋与成就系统

- 系统包含 3 名学生智能体。每个智能体随机获得一种初始天赋（Trait），用于推动行为差异化：
  - 夜猫子：22:00 - 02:00 期间行动，心情值不降反升。
  - 干饭王：在食堂进食时额外恢复 10 点饱食度。
  - 体弱多病：初始体力上限为 80。
- 当智能体在某一领域累计投入达到阈值时，将解锁永久成就 Buff：
  - 内卷之王：累计自习 30 小时后解锁；此后每次自习额外 +5 学分。
  - 运动健将：累计运动 15 小时后解锁；此后体力上限提升至 150。
  - 时间管理大师：连续 48 小时未触发任何 Debuff 后解锁；此后所有动作的体力消耗减少 20%。

终局目标

课程设计的总体目标是在 168 小时内最大化团队总得分。团队总得分可定义为：

代码块

```
总得分 = Σ(学分 × 2.0 + 健康值 × 1.5 + 心情值 × 1.0 + 解锁成就数 × 100)。
```

学生需围绕这一目标设计规则策略、搜索算法和 LLM 驱动策略。

阶段一：环境搭建、游戏设计与行为可视化

面向对象建模与规则引擎

- 使用 OOP 思想封装 Map、Building、Agent、TimeSystem、Achievement 等核心类。
- 实现时间推进、营业时间检查、容量限制检查、动作执行与属性结算。
- 系统应能够正确处理非法动作，例如：在 23:00 前往图书馆、到达已满员的体育馆、在未到达目标建筑时提前执行动作等。

行为可视化要求

第一阶段必须提供可视化展示能力，用于观察和调试智能体行为。命令行版本为基础要求，GUI 版本为进阶实现。

- 基础要求（必做）：采用命令行可视化。每个时间步至少输出当前日期/小时、各智能体所在位置、正在执行的动作、体力、饱食度、学分、健康值、心情值、当前 Debuff 和已解锁成就。
- 进阶要求（选做）：设计图形化界面（GUI），以网格地图或节点图方式展示建筑位置和智能体移动过程，可实时刷新或按步播放。
- 无论使用命令行还是 GUI，都应使观察者能够直观看到“从哪里出发、移动到哪里、用了多久、到达后做了什么”。

规则驱动的 Baseline 调度器

- 学生需实现一个纯代码驱动的自动化排班 AI（不能依赖纯随机），作为第一阶段的 Baseline。该调度器应能够在完整 168 小时内持续运行，并尽量避免触发“疲劳”和“饥饿”等负面状态。
- Baseline 不应只按收益高低机械选择动作，而应同时考虑建筑距离、移动耗时、营业时间和容量约束。例如：在饱食度较低时优先选择“来得及到达且正在营业的食堂”；在图书馆收益虽高但距离过远或即将关门时，应主动放弃该目标。

阶段二：启发式搜索规划与 LLM 双引擎驱动

本阶段保持原有分工不变：在第一阶段已完成的游戏环境基础上，废弃简单的贪心规则，引入经典状态空间搜索算法，并结合 LLM 构建“高层认知 + 底层精确规划”的混合智能体。

状态空间搜索与时域规划算法

- 滚动时域搜索（Rolling Horizon Search）：每个时间步向前展开并模拟未来 6 - 12 小时的动作组合，选择预测窗口末期综合价值最高的路径，仅执行第一步。
- 束搜索（Beam Search）：设定宽度 K，在未来日程推演时每层只保留评价最高的 K 个分支。
- 蒙特卡洛树搜索（MCTS）：通过多次随机模拟估计当前动作的长期价值。

学生至少实现上述一种规划算法，并将“地图距离 - 移动耗时 - 营业时间 - 容量约束 - Debuff 风险 - 成就进度”共同纳入搜索状态。

启发式评估与时空剪枝

- 设计启发式评估函数 H(s)，不仅要考虑当前学分、健康值与心情值，还要评估体力、饱食度安全边界以及成就进度的潜在价值。
- 结合时空信息进行剪枝。例如：若目标建筑距离过远、到达后将超过营业时间，或移动后会触发严重 Debuff，则可直接剪去该分支。
- 在有容量限制的场景中，应考虑多个智能体到达同一资源点的冲突风险。

引入 LLM 进行人设驱动与动态调优

- 根据智能体当前状态、初始天赋和自然语言目标，让 LLM 动态输出或调整启发式函数中各指标的权重。

- 当未来预测中出现图书馆或体育馆资源冲突时，可调用 LLM 生成协商建议，再将协商结果转化为规划约束。
- 鼓励学生比较“无 LLM 的纯算法策略”和“LLM + 规划算法”的策略差异。